

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

A)10

Hello

B)x

display

C)Only Hello

D)Error

Q2. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

A)TypeError

B)20

C)30

D)SyntaxError

Q3. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B

B)A C

C)B C

D)A

Q4. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)Nothing

B)A

C)B

D)A and B

Q5. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)20

B)NameError

C)None

D)10

Q6. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A)ValueError

B)Nothing

C)e

D)Wrong Value

Q7. Which object keeps track of modules that have already been



imported during the current Python process?

- A)sys.path
- B)sys.imports
- C)sys.modules
- D)__module__

Q8. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

- A)None
- B)5
- C)6
- D)3

Q9. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

print(x, y)

- A)15 5
- B)5 5
- C)15 15
- D)5 15

Q10. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents syntax errors
- B)It identifies code intended to run when the file is executed directly
- C)It prevents all imports
- D)It creates the main function

Q11. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went
```

wrong")

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

- A)TypeError
- B)Something went wrong
- C)Exception
- D)MyError

Q12. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

print(fun())

- A)10 followed by Finally
- B)Finally followed by 10
- C)Finally only
- D)10 only

Q13. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

print(result)

- A)[2, 4]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[4, 8]

Q14. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)TypeError
- B)0



C)NameError

D)None

Q15. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)Hello

B>Error

C)Hello None

D)None

Q16. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

1

3

B)1

2

3

4

C)1

1

1

1

D)0

1

0

2

Q17. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)20

B)30

C)TypeError

D)SyntaxError

Q18. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A)except blocks execute randomly

B)ZeroDivisionError is not an exception

C)Exception is a superclass of ZeroDivisionError

D)ZeroDivisionError occurs before Exception

Q19. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)None

B)15

C)10

D)NameError

Q20. What is the output?

```
def outer():
```

```
    x = 10
```



```
def inner():  
    nonlocal x  
    x = x + 5  
    return x
```

```
inner()  
return x
```

```
print(outer())
```

- A)5
- B)UnboundLocalError
- C)15
- D)10

Q21. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)InvalidAgeError
- 15
- B)InvalidAgeError
- Age is too low
- C)Nothing, because custom exceptions cannot be caught
- D)Exception
- Age is too low

Q22. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)10
- B)15
- C)5
- D>Error

Q23. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A)First imports only sqrt
- B)Both import only sqrt
- C)There is no difference
- D)First provides the module namespace; second imports sqrt directly

Q24. What will normally happen?

```
import math  
import math
```

- A)The module is normally loaded and executed twice
- B)ImportError
- C)SyntaxError
- D)The module is normally loaded once and reused

Q25. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

- A)A
- B
- D
- 10
- B)A
- C
- D
- 20



C)A
B
C
D
10
D)A
B
10
D

Q26. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)B
B)A B
C)Nothing
D)A

Q27. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

A)It executes correctly
B)TypeError
C)SyntaxError
D)NameError

Q28. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

A)RuntimeError
B)20
C)10
D)None

**Q29. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

A)B
B)Nothing
C)A
D)A followed by B

Q30. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

A)A C
B)A
C)B
D)B C

Q31. What will be printed?

from functools import reduce

numbers = [1, 2, 3, 4]

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

A)9
B)10
C)24
D)0

Q32. What will be the output?

x = 10

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

inner()



print(x)

**outer()
print(x)**

A)20

10

B)30

30

C)20

30

D)30

20

Q33. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

**import module_a
import module_a**

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)10

10

C)ImportError

D)Module loaded

Module loaded

10

Q34. What happens when this code runs?

**def first():
 second()**

**def second():
 third()**

**def third():
 raise RuntimeError("Failure")**

try:

first()

except RuntimeError:

print("Handled")

A)TypeError

B)Handled

C)NameError

D)RuntimeError cannot propagate through functions

Q35. What will be the output?

def fun(x):

try:

if x < 0:

raise ValueError("negative")

return x

except ValueError:

return 100

finally:

print("cleanup")

print(fun(-5))

A)cleanup

100

B)negative

cleanup

C)100

cleanup

D)cleanup

-5

Q36. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()

print(a)

print(b)

A)[10]

[10]

B)[10, 10]

[10]

C)Error

D)[10]

[10, 10]



Q37. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)20

B)10

C)30

D)NameError

Q38. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)Error

B)10

C)func(5)

D)5

Q39. What is the output?

```
def fun(a=10, b=20):
```

```
    return a - b
```

```
print(fun(30))
```

A)30

B)10

C)20

D)-10

Q40. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, followed by propagation of the same exception

B)No Output

C)B

D)A, and execution ends normally

Q41. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A B

B)A

C)SyntaxError

D)B

Q42. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)20

C)NameError

D)10

Q43. What will happen?

```
def fun(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * fun(n)
```

```
print(fun(3))
```

A)3

B)TypeError

C)6

D)RecursionError

Q44. What will happen?

```
def fun(a, b=10):
```

```
    return a + b
```



print(fun(b=5, a=20))

- A)15
- B)25
- C)SyntaxError
- D)TypeError

Q45. What happens here?

from math import sqrt

print(sqrt(36))

- A)6.0
- B)6
- C)NameError
- D)36

Q46. What will be the output?

import math

print(math.sqrt(25))

- A)5.0
- B)5
- C)25
- D)sqrt(25)

Q47. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)The local file can shadow the intended standard-library module
- B)Python will always reject it
- C)random cannot be imported
- D)Python automatically renames the local file

Q48. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

- A)UnboundLocalError
- B)10
- C)50
- D)NameError

Q49. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)NameError
- B)20
- C)10
- D)UnboundLocalError

Q50. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)Nothing
- B)A
- C)A B
- D)B

